



```
In [1]: from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense
        from tensorflow.keras.models import Sequential
        from tensorflow.keras.datasets import mnist
        import matplotlib.pyplot as plt
        import numpy as np
        from sklearn.metrics import confusion_matrix
        import seaborn as sns

        # Load dataset
        (X_train, y_train), (X_test, y_test) = mnist.load_data()

        # Normalize
        X_train = X_train / 255.0
        X_test = X_test / 255.0

        # Reshape for CNN
        X_train = X_train.reshape(-1,28,28,1)
        X_test = X_test.reshape(-1,28,28,1)

        # Model
        model = Sequential([
            Conv2D(32, (3,3), activation='relu', input_shape=(28,28,1)), # 1 Convolu
            MaxPooling2D((2,2)), # 1 Poolin
            Flatten(),
            Dense(128, activation='relu'), # Dense 1
            Dense(10, activation='softmax') # Dense 2
        ])

        model.compile(optimizer='adam',
                      loss='sparse_categorical_crossentropy',
                      metrics=['accuracy'])

        model.summary()

        # Train
        history = model.fit(X_train, y_train,
                            epochs=10,
                            batch_size=64,
                            validation_split=0.1)

        # Test accuracy
        test_loss, test_acc = model.evaluate(X_test, y_test)
        print("Test accuracy:", test_acc)
```

Downloading data from <https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz>

11490434/11490434 ————— 0s 0us/step

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
flatten (Flatten)	(None, 5408)	0
dense (Dense)	(None, 128)	692,352
dense_1 (Dense)	(None, 10)	1,290

Total params: 693,962 (2.65 MB)

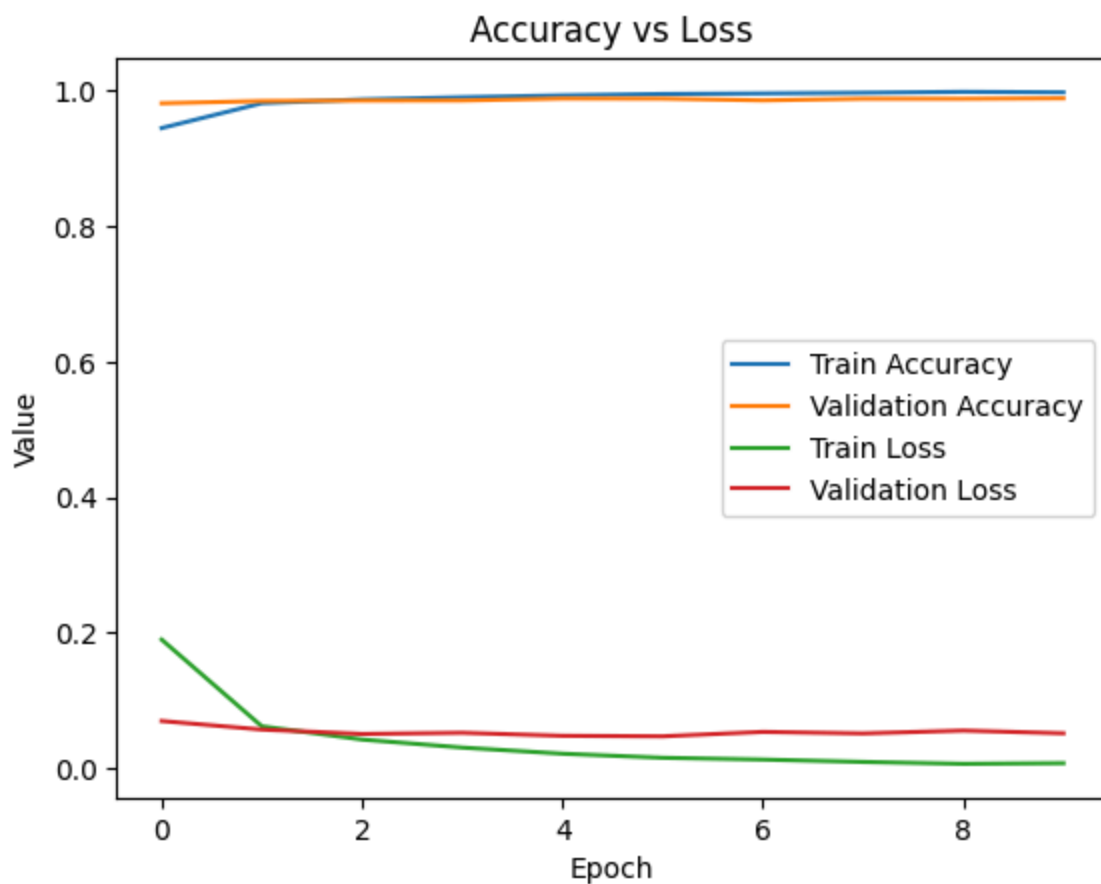
Trainable params: 693,962 (2.65 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/10
844/844 ————— **8s** 5ms/step - accuracy: 0.9449 - loss: 0.1896 - val_accuracy: 0.9815 - val_loss: 0.0694
Epoch 2/10
844/844 ————— **2s** 3ms/step - accuracy: 0.9816 - loss: 0.0615 - val_accuracy: 0.9848 - val_loss: 0.0567
Epoch 3/10
844/844 ————— **3s** 3ms/step - accuracy: 0.9872 - loss: 0.0420 - val_accuracy: 0.9855 - val_loss: 0.0502
Epoch 4/10
844/844 ————— **4s** 4ms/step - accuracy: 0.9904 - loss: 0.0301 - val_accuracy: 0.9857 - val_loss: 0.0520
Epoch 5/10
844/844 ————— **3s** 3ms/step - accuracy: 0.9932 - loss: 0.0211 - val_accuracy: 0.9885 - val_loss: 0.0475
Epoch 6/10
844/844 ————— **3s** 3ms/step - accuracy: 0.9952 - loss: 0.0151 - val_accuracy: 0.9883 - val_loss: 0.0468
Epoch 7/10
844/844 ————— **3s** 3ms/step - accuracy: 0.9960 - loss: 0.0124 - val_accuracy: 0.9858 - val_loss: 0.0533
Epoch 8/10
844/844 ————— **3s** 4ms/step - accuracy: 0.9971 - loss: 0.0089 - val_accuracy: 0.9882 - val_loss: 0.0510
Epoch 9/10
844/844 ————— **4s** 3ms/step - accuracy: 0.9984 - loss: 0.0061 - val_accuracy: 0.9882 - val_loss: 0.0552
Epoch 10/10
844/844 ————— **3s** 3ms/step - accuracy: 0.9977 - loss: 0.0071 - val_accuracy: 0.9888 - val_loss: 0.0512
313/313 ————— **2s** 4ms/step - accuracy: 0.9863 - loss: 0.0475
Test accuracy: 0.986299991607666

```
In [2]: plt.figure()
plt.plot(history.history['accuracy'], label='Train Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation Accuracy')
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')

plt.xlabel("Epoch")
plt.ylabel("Value")
plt.title("Accuracy vs Loss")
plt.legend()
plt.show()
```



```
In [3]: y_pred = model.predict(X_test)
y_pred_classes = np.argmax(y_pred, axis=1)

cm = confusion_matrix(y_test, y_pred_classes)

plt.figure()
sns.heatmap(cm, annot=True, fmt='d')
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

